7-4 · MinIO·Qdrant 연동 · 실습 1H

MinIO 버킷 구성과 문서 적재 실습

rag-minio 를 띄우고 버킷·객체를 다룬다

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① rag-minio 를 볼륨과 함께 띄우고 웹 콘솔(9001)에 로그인한다

② 접속은 minio_env.py 한 곳에 두고 raw-docs 에 두 문서를 올린다

③ 일부러 secure=True 로 붙어 SSL 오류를 직접 본다

④ 읽기 · 없는 키 · 접두사 — manual/guide.txt 까지 세 문서를 채운다

사전 준비: 실습점검 1 · 2단계 — qdrant 가 Up 이고 9000 · 9001 이 비어 있으면 된다

실습 흐름

여섯 단계 — 파일을 만들고 python 으로 돌린다

cat > 파일.py << 'EOF' ... EOF → python 파일.py (6주차와 같은 방식)

1. rag-minio	2. 부품 · 업로드	3. 일부러 실패	4. 읽기	5·6. 없는 키 · 접두사
docker run 콘솔 9001	minio_env.py mc_upload.py	secure=True SSLError	read_text 642 자	NoSuchKey manual/ 추가

1단계

docker run 한 줄 — 옵션마다 뜻

옵션	뜻
docker run -d --name rag-minio	백그라운드로, 이 주차 전용 이름
-p 9000:9000 -p 9001:9001	9000 = API · 9001 = 웹 콘솔
-e MINIO_ROOT_USER=minioadmin	로그인 계정 = 코드의 access_key
-e MINIO_ROOT_PASSWORD=minioadmin	비밀번호 = 코드의 secret_key
-v rag_minio_data:/data	버킷 · 문서를 담는 볼륨
pgsty/minio server /data --console-address ":9001"	서버 실행 · 콘솔 주소 지정

확인 포인트: http://localhost:9001 에 minioadmin / minioadmin 으로 로그인되면 성공

docker ps 에 rag-minio · Up · 0.0.0.0:9000-9001(실측 1초 안에) · 다음 날은 docker start rag-minio

2단계 · 부품

접속 정보는 minio_env.py 한 곳에

1. **cat > minio_env.py << 'EOF' ... EOF**
   출력 없음 — 7-4 · 7-5 가 함께 불러 쓴다

2. **mc = Minio("localhost:9000", ..., secure=False)**
   로컬 컨테이너는 HTTP — 키는 둘 다 minioadmin

3. **BUCKET = "raw-docs"**
   오늘 쓰는 버킷 이름도 여기 한 곳

4. **def read_text(bucket, key)**
   get_object → decode · finally 에서 close() · release_conn()

7-5 에서도

같은 파일을

불러 쓴다

2단계 · 올리기

버킷을 만들고 7-2 의 두 문서를 올린다

1. **if not mc.bucket_exists(BUCKET): mc.make_bucket(BUCKET)**
   두 번째 실행에도 오류가 없게
2. **mc.fput_object(BUCKET, "notes/notes.txt", "notes.txt")**
   long_notes.txt 도 notes/ 아래로
3. **python mc_upload.py**
   notes/long_notes.txt 2945 bytes · notes/notes.txt 1506 bytes(실측)

바이트 수

2945 · 1506

> **7-2 의 wc -m 은 글자 수(1,247 · 642) — 한글 한 글자가 UTF-8 로 3바이트라 두 배 넘게 크다**

3단계 — 일부러 실패시키기

secure=True 로 붙어 본다 — 한 줄이 몇 시간을 먹는다

secure=False — 로컬 컨테이너

HTTP 로 말하는 서버에 HTTP 로

- minio_env.py 의 mc 가 이것
- bucket_exists 가 답을 준다
- 운영 HTTPS 에서는 기본값(True)

VS

secure=True — python mc_https.py

재시도 때문에 몇 초 걸린 뒤 예외(실측)

- MaxRetryError
- Caused by SSLError(...)
- [SSL: WRONG_VERSION_NUMBER]

4·5단계

읽어 오고, 없는 키를 읽어 본다

1. **python mc_read.py — read_text(BUCKET, "notes/notes.txt")**
   642 자 · 「1장 데이터 파이프라인 개요」로 시작(실측)

2. **finally 의 close() · release_conn()**
   오류가 나도 연결을 돌려준다 — 반복문에서 빼면 곧 고갈

3. **python mc_nokey.py — "notes/없는파일.txt"**
   S3Error - NoSuchKey(실측)

4. **실제 키 목록을 찍는다**
   ['notes/long_notes.txt', 'notes/notes.txt'] — 한글 키는 NFC/NFD 로 어긋난다

객체 키는

영문으로

통일

6단계

manual/ 을 더하고 접두사로 범위를 좁힌다

전체 — recursive=True

guide.txt(장애 대응 수칙)를 manual/ 에 올려 셋

- manual/guide.txt — ERR-4021 · 5003 · 5017
- notes/long_notes.txt
- notes/notes.txt

VS

prefix="notes/" — 범위 좁히기

notes/ 아래 둘만 나온다(실측)

- notes/long_notes.txt
- notes/notes.txt
- 종류 · 부서 · 연도별 접두사 → 「이 폴더만 재색인」

확인 포인트

이 1시간이 끝날 때 확인할 것

1. rag-minio 가 Up 이고 localhost:9001 에 로그인된다
2. mc_upload.py 가 2945 · 1506 bytes 두 줄을 낸다
3. secure=True 로 붙으면 MaxRetryError · SSLError
4. mc_read.py 가 642 자와 「1장 데이터 파이프라인 개요」를 낸다
5. 없는 키는 S3Error - NoSuchKey
6. 전체는 세 문서, prefix="notes/" 는 두 문서

7-4 · 실습 1H 마무리

다음은 MinIO → 임베딩 → Qdrant 파이프라인 (1.5H)

버킷을 훑어 자르고 임베딩해 lecture_kb 에 넣는 kb.py 를 만들고, 재실행해도 늘지 않는지 assert 로 잽니다

- ✓ **kb.py — ingest() 로 세 문서 10조각**
- ✓ **재실행 10 → 10 을 assert 로**
- ✓ **uuid4 로 바꿔 3 → 6 재현**
- ✓ **source 역참조 · 줄어든 문서 재적재**

가져갈 것

minio_env.py · raw-docs 세 문서